CONNECTOR: Enhancing the Traceability of Decentralized Bridge Applications via Automatic Cross-chain Transaction Association

Dan Lin
Sun Yat-sen University

Jiajing Wu
Sun Yat-sen University

Yuxin Su
Sun Yat-sen University

Ziye Zheng
Sun Yat-sen University

Yuhong Nan
Sun Yat-sen University

Qinnan Zhang
Beihang University

Bowen Song
Ant Group

Zibin Zheng
Sun Yat-sen University

Abstract

Decentralized bridge applications are important software that connects various blockchains and facilitates cross-chain asset transfer in the decentralized finance (DeFi) ecosystem which currently operates in a multi-chain environment. Cross-chain transaction association identifies and matches unique transactions executed by bridge DApps, which is important research to enhance the traceability of cross-chain bridge DApps. However, existing methods rely entirely on unobservable internal ledgers or APIs, violating the open and decentralized properties of blockchain. In this paper, we analyze the challenges of this issue and then present CONNECTOR, an automated cross-chain transaction association analysis method based on bridge smart contracts. Specifically, CONNECTOR first identifies deposit transactions by extracting distinctive and generic features from the transaction traces of bridge contracts. With the accurate deposit transactions, CONNECTOR mines the execution logs of bridge contracts to achieve withdrawal transaction matching. We conduct real-world experiments on different types of bridges to demonstrate the effectiveness of CONNECTOR. The experiment demonstrates that CONNECTOR successfully identifies 100% deposit transactions, associates 95.81% withdrawal transactions, and surpasses methods for CeFi bridges. Based on the association results, we obtain interesting findings about cross-chain transaction behaviors in DeFi bridges and analyze the tracing abilities of CONNECTOR to assist the DeFi bridge apps.

1 Introduction

Decentralized applications (DApps) on the blockchain combine backend smart contracts with frontend user interfaces, allowing users to participate in various decentralized finance (DeFi) activities without the need to trust centralized third-party institutions [22]. Decentralized bridges (referred to as *DeFi bridges*) are a type of DApp that provides cross-chain transaction services. Currently, the blockchain ecosystem consists of multiple chains, with records of 260 public blockchains as of March 2024, according to data from DeFiLlama[1]. However, data between different blockchain systems are not interoperable, akin to isolated islands. Therefore, DeFi bridges, as applications capable of facilitating asset circulation and information exchange between chains, can promote further development of the multi-chain ecosystem [11].

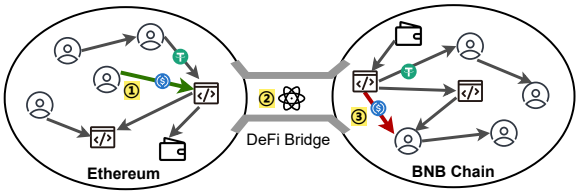


Figure 1: A user making a cross-chain transaction from Ethereum to BNB chain includes three main steps: ① User locks assets to bridge contract on Ethereum (deposit transaction tx_{dep}); ② The bridge verifies the transaction; ③ The bridge contract unlocks the corresponding value of the asset to the user on the BNB chain (withdrawal transaction tx_{wth}).

Decentralized bridges do not actually connect the ledgers and assets between different blockchains but rather serve as intermediaries. From the user's view, completing a cross-chain transaction via a DeFi bridge actually consists of two parts: a deposit transaction on the source chain and a withdrawal transaction on the destination chain. Fig. 1 shows the process of a user who wants to transfer his UDSC coins from his Ethereum account to his BNB chain account. By the idea of depositing money on the source chain and withdrawing it on the destination chain, bridge DApps enable crypto assets to be used on different blockchains.

Monitoring the users' behavior and transactions on DeFi bridges is crucial. Specifically, this paper explores the issue of ***cross-chain transaction association*** (as described in §4), which refers to identifying and matching the unique and consistent transactions executed by bridge applications on the source and destination chains to obtain cross-chain transaction pairs. For developers of bridge DApps, firstly, cross-

[1] https://defillama.com/chains

chain transaction association can enhance the traceability of bridge applications [3], preventing illicit activities such as money laundering by obfuscating and transferring illegal assets through bridge DApps, thereby helping developers reduce compliance risks. According to Elliptic's report [4], Ren Bridge has been used for money laundering involving over $540 million worth of illegal cryptocurrency assets. Furthermore, the lack of effective cross-chain transaction association analysis methods may fail in detecting token leakage attacks of bridge DApps [15]. For ordinary users, this can improve the experience of bridge users. In conclusion, research on cross-chain transaction association can enhance the reliability and usability of bridge applications themselves.

Compared to other DApps, DeFi bridges have unique characteristics: 1) Complexity: DeFi bridges require multi-step interactions of contracts to fulfill multiple business requirements, and these contracts have richer types of functionalities, such as `Lock`, `Confirm`, `Send` etc; 2) Isolation: Multiple contracts of the DeFi bridges need to be deployed separately on mutually isolated source and destination chains for distributed execution. Deposit transactions on the source chain and withdrawal transactions on the destination chain are segregated, as illustrated in Fig. 1. Therefore, cross-chain transaction association faces two main challenges (C):

- **C1: Identify complex deposit transactions on the source chain.** The transaction volume of bridge contracts is large, but users' cross-chain deposit transactions are only a part of them and are not explicitly marked on the blockchain. Existing pattern-based transaction identification methods cannot adapt well to complex and diverse cross-chain scenarios.
- **C2: Match isolated withdrawal transactions on the destination chain.** Due to the isolation characteristics of cross-chain bridge DApps, the process of cross-chain transaction pairs is not fully recorded on any single blockchain ledger, and the relation between source and destination chains is not explicitly recorded on the chain. Therefore, known source chain transactions of cross-chain transactions do not directly get corresponding transactions on destination chains.

In this paper, we propose a novel analysis approach named CONNECTOR to address **cr**oss-chain tra**n**saction associatio**n** in d**ec**en**t**ralized bridge c**o**nt**r**acts. We first conduct an empirical study of bridge DApps and summarize the *cross-chain transaction metadata* as the essential elements for cross-chain transaction actions. To tackle **C1**, we combine various aspects of the bridge's input data and execution trace data, extracting semantic and structural features of transaction intent to jointly characterize the intent of cross-chain deposit transactions. To tackle **C2**, we analyze the message-passing mechanism of the bridge at the contract log level, parsing logs for syntax and semantics to extract cross-chain transaction metadata, enabling precise matching in massive transaction data. To evaluate the effectiveness of CONNECTOR, we collect the cross-chain transaction pairs from open-source DeFi bridges. We build the first dataset with 24,392 cross-chain transaction pairs from bridges of different cross-chain mechanisms. We conduct experiments on this dataset and successfully identify 100% deposit transactions and associate 95.81% withdrawal transactions. Furthermore, based on transaction pairs output by CONNECTOR, we conduct an empirical study in §7 to analyze the characteristics of identified deposit transactions, as well as the matched withdrawal transactions to analyze the issues including the user privacy protection, time efficiency, and economic benefits on the cross-chain bridges. In summary, our main contributions are as follows.

- **Problem:** To the best of our knowledge, we are the *first* work on cross-chain transaction association tool for decentralized bridge applications. We design CONNECTOR[2] with a more general, multi-cryptocurrency enabled and multi-blockchain-adapted setting.
- **Evaluation:** CONNECTOR automatically identify deposit transactions with 100% accuracy and associate withdrawal transactions with a 95.81% matching rate. We also apply CONNECTOR to compare with bridge explorers and CONNECTOR is validated on 50 unlabelled pairs.
- **Analysis:** Based on identified deposit transactions and matched withdrawal transactions, we obtain interesting findings through statistical analysis, feature analysis, and graph analysis. Particularly, we find that it is common for ordinary users to use the same address in cross-chain transactions, and DeFi bridges are used in crypto money laundering.

2 Background

2.1 Blockchain and Transactions

Permissionless blockchains are independent, and their respective ledgers are not interconnected. Ethereum is the first blockchain to support smart contracts, running on the Ethereum Virtual Machine (EVM). Ethereum has two types of accounts: external owned accounts (EOA) - controlled by anyone with the private key, and contract accounts - smart contracts deployed on the network controlled by code. Smart contracts [7, 8] include code data that can be executed automatically on blockchains. The smart contract code writing in Solidity language [5] can be compiled to generate binary code (bytecode), and also generate the application binary interface (ABI). The ABI records all the functions and events provided by the contract and the parameters that should be passed. There are two types of transactions: external transactions - initiated by EOA, and internal transactions - initiated by smart contracts. An external transaction can trigger multiple internal transactions. During the execution of a transaction, the smart contract may trigger defined events into log data. Smart contract events are a special type of structure and are recorded in the EVM's logs.

[2]Available at https://github.com/Connector-Tool.

2.2 DeFi Bridges vs. CeFi Bridges

Based on the adopted trust and validation models, cross-chain bridge apps can currently be categorized into centralized bridges (*CeFi bridges*) and decentralized bridges (*DeFi bridges*) Fig. 2 illustrates the comparison of the CeFi bridge and DeFi bridges, which have different implementation mechanisms and carriers.

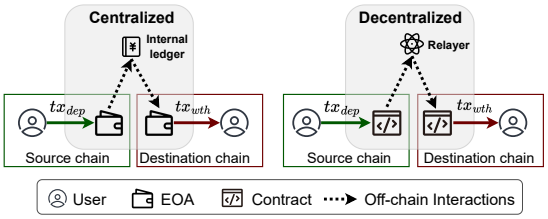


Figure 2: CeFi bridges vs DeFi bridges.

CeFi bridge: Most CeFi bridge apps are externally owned accounts (EOA)-based without on-chain code, similar to centralized exchanges (CEX) [25]. In CeFi bridges, digital assets, and transaction data for cross-chain services are custodied by centralized entities, e.g., ShapeShift [23]. The centralized entities behind these bridges maintain an internal ledger that records how assets from the source chain are transferred to the destination chain.

DeFi bridge: DeFi bridge apps are primarily contract-based DApps [24], similar to decentralized exchanges. The on-chain router contract is responsible for interacting with users and various token contracts, providing on-chain functionality, including locking and unlocking users' tokens, and recording token transfer information as specific on-chain events for off-chain relayers to access. Correspondingly, off-chain relayers are responsible for fetching on-chain events from the source chain and coordinating with router contracts on the target chain to complete cross-chain asset transfers.

3 Data Source and Statistics

Current research on cross-chain primarily focuses on blockchain interoperability protocols [14] or generic cross-chain architectures, which are orthogonal to application-level bridge applications we discuss. Thus, we try to address a gap in empirical research by examining application-level cross-chain bridges within the market, particularly the trust model (i.e., centralization vs. decentralization) and traceability performance. We first investigate the websites that currently contain bridge apps, including DefiLlama, Chainspot, and DeFi Rekt Database. Chainspot[3] currently records the highest number of bridges. According to Chainspot's statistics, there are currently 120 recorded bridges. We initiate an empirical study to assess the traceability of these bridges, examining various aspects, including their official websites, open-source documentation, verification mechanisms, contract addresses and codes, and transaction retrieval services.

Regarding the official websites of these cross-chain bridges, 17.50% bridges do not provide an official website or have discontinued their use. It is important to note that this specific portion of the bridge will not be factored into the subsequent statistics. Regarding open-source documentation, 19.34% bridges do not offer white paper or design documentation. Regarding the verification mechanisms, 4.08% bridges are validated by a centralized party, which we considered as CeFi bridges and 89.80% bridges are validated by multi-validation with trustless code. The remaining 6.12% of bridges cannot validate the validation mechanism due to the lack of documentation on the implementation architecture. Among the contract-based DeFi bridges, 81.63% of bridges disclose their contract addresses and codes.

Among the available bridges that provided official websites, 67.35% bridges do not offer cross-chain transaction tracking services or bridge explorers[4]. 19.39% bridges offer cross-chain transaction query services but do not provide detailed cross-chain information. Just **13.27% of bridges (13 in total)** offer full cross-chain transaction tracking services, i.e., providing users' deposit and withdrawal transactions. There could be several reasons why cross-chain bridges do not offer transaction tracking services: a lack of focus and investment in cross-chain transaction tracking services; limited by the technical complexity and resource constraints; security and privacy considerations. The comprehensive dataset can be accessed at the open repository. Among the remaining bridges that do claim to provide tracking services, we observe that they may not consistently return accurate results. For example, in Celer cBridge, a user's cross-chain transaction on Ethereum (deposit transaction hash *0x13b1*) could not be correctly associated with the withdrawal transaction using the provided service (See §6.2.2 for more discussion).

4 Problem Statement and Definition

As described in §3, the traceability of bridge applications refers to the ability to track asset movements and transaction activities after using the bridge apps. Our research focuses on the cross-chain transaction association problem, specifically, identifying and matching the cross-chain transaction pairs of a DeFi bridge between different blockchain platforms.

PROBLEM (CROSS-CHAIN TRANSACTION ASSOCIATION): *Given a DeFi bridge without centralized ledgers or APIs, we suppose it supports K different and non-interconnected blockchain platforms. The number of the bridge transaction*

[3] https://chainspot.io/

[4] It is worth noting that two potentially confusing scenarios do not qualify as providing cross-chain transaction association services: 1) offering a blockchain explorer instead of a bridge explorer, e.g. Avalanche Bridge; 2) providing an explorer for the latest transactions without supporting history retrieval, e.g. Across.

data on the k-th chain is N_k, and the transaction set in its ledger is denoted as $TX^k = \{tx_1^k, tx_2^k, ..., tx_{N_k}^k\}$. Each transaction is uniquely identified by its transaction hash.

- *Identify the deposit transaction tx_i^A on the source chain A;*
- *Locate corresponding withdrawal transaction tx_j^B on destination chain B for each deposit transaction.*

This process ultimately results in a collection of cross-chain transaction pairs with consistent one-deposit and one-withdrawal behavior[5], denoted as $\{(tx_i^A, tx_j^B) | tx_i^A \in TX^A, tx_j^B \in TX^B\}$. For the sake of subsequent description, we first define ***cross-chain transaction metadata*** (denoted as $\mathbb{M}$) for a cross-chain transaction pair, as shown in Table 1.

Table 1: Cross-chain transaction metadata ($\mathbb{M}$). Superscript 's' indicates source chains, and 'd' indicates destination chains.

Symbol	Description	Symbol	Description
$\mathbb{M}.txhash^s$	Deposit transaction hash	$\mathbb{M}.txhash^d$	Withdrawal transaction hash
$\mathbb{M}.chain^s$	Source chain	$\mathbb{M}.chain^d$	Destination chain
$\mathbb{M}.sender$	Deposit address	$\mathbb{M}.receiver$	Withdrawal address
$\mathbb{M}.asset^s$	Deposit asset hash	$\mathbb{M}.asset^d$	Withdrawal asset hash
$\mathbb{M}.amount^s$	Deposit amount	$\mathbb{M}.amount^d$	Withdrawal amount
$\mathbb{M}.timestamp^s$	Deposit timestamp	$\mathbb{M}.timestamp^d$	Withdrawal timestamp

Cross-chain transaction metadata constitutes the essential data for cross-chain transaction actions in bridge DApps, regardless of the variant mechanisms of bridges, which can be generalized to all types of bridges. Cross-chain transaction metadata comprises two sets of information from the source and destination chains. The metadata from each chain includes fundamental details of a transaction, such as transaction hash, user address, timestamp, asset hash (i.e., type), and amount. This metadata is the key to cross-chain transaction identification and matching. Our design idea of CONNECTOR is to reveal the metadata of cross-chain transactions reliant on bridge smart contracts. The proposed cross-chain metadata are deeply involved in the subsequent deposit transaction identification (See §5.1) and withdrawal transaction identification (See §5.2).

4.1 Existing Work and Limitations

The comparison of existing cross-chain transaction association methods and ours are shown in Table 2. Yousaf, Kappos, and Meiklejohn (referred to as YKM Heuristics hereafter) [23] is the first to study the cross-chain transaction association problem in EOA-based CeFi bridges. Zhang *et al.* [25] devise CLTracer, a framework grounded in address relationships for cross-chain transaction clustering. Existing methods cannot be directly applied to DeFi bridges without centralized ledgers or APIs, and still have the following research gaps: *1) Low accuracy:* The accuracy of the original tracing algorithm proposed in the YKM Heuristics, which relies on bridge APIs to identify deposit transactions on the blockchain, is only 80% [23], leaving room for improvement. *2) Non-independent:* Existing methods rely entirely on the internal APIs to match withdrawal transactions. This leads to a lack of user confidence in the traceability of bridges, as internal data can be manipulated by a centralized entity with the potential for tampering and not aligning with the nature of DeFi. This motivates us to design cross-chain transaction association method for DeFi bridges.

[5] Inconsistent behavior like attack transactions [24] and many-to-many cross-chain relationships are out of the scope discussed in this paper.

Table 2: Comparison of existing related works.

Solutions	Supported Types	Bridge Carriers	Data Source	ERC20 Assets?	Without Bridge APIs?	Support History?
YKM [23]	CeFi Bridge Shapeshift	EOA	Synchronized full ledgers	×	×	×
CLTracer [25]	CeFi Bridge/ CEX	EOA	Synchronized full ledgers	×	×	✓
Ours	DeFi Bridge	Contract	Crawling limited data	✓	✓	✓

5 Methodology of CONNECTOR

In this section, we design CONNECTOR to address cross-chain transaction association for DeFi bridges. As shown in Fig. 3, CONNECTOR consists of two main steps. CONNECTOR takes the transaction data, traces, and logs from the cross-chain bridge contract as input and outputs the bridge's transaction pairs. The first step (**S1: Deposit Transaction Identification**) extracts effective features by mining input data and token-aware call relationships of contract transactions for deposit behavior identification. The second step (**S2: Withdrawal Transaction Matching**) parses the contract log from both syntactic and semantic view, and match the withdrawal transaction via business logic.

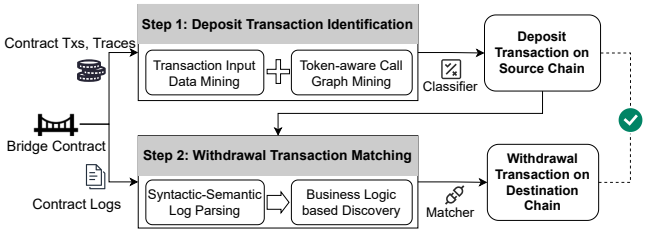


Figure 3: Overview of CONNECTOR.

5.1 Deposit Transaction Identification

In bridge applications, the first step for users in conducting cross-chain transactions typically involves initiating a deposit transaction on the source chain. Only by knowing the deposit transactions on the source chain can we further match the corresponding withdrawal transactions. The purpose is to intelligently and precisely distinguish transactions that belong to cross-chain deposit activities, ensuring the effectiveness and efficiency of subsequent cross-chain transaction correlation. Existing methods identify transactions by manually

```
//Poly Network
function lock(address fromAsset, uint64 toChainId, bytes
    toAddress, uint256 amount, uint256 fee, uint256 id);
//Celer cBridge
function send(address _receiver, address _token, uint256
    _amount, uint64 _dstChainId, uint64 _nonce, uint32
    _maxSlippage);
function sendNative(address _receiver, uint256 _amount,
    uint64 _dstChainId, uint64 _nonce, uint32
    _maxSlippage);
//Multichain
function anySwapOutNative(address token, address to,
    uint256 toChainID);
```

Figure 4: Example deposit functions defined in Poly Network, Celer cBridge, and Multichain.

pre-collecting function names keywords and conduct corresponding method ID database for lookup [9,26]. However, the major challenge with this approach lies in its lack of generality, which can lead to false positives. 1) It struggles to handle newly added functions during bridge contract upgrades; 2) It faces difficulties in adapting to all diverse bridge contracts.

Deposit transactions in DeFi bridges are implemented through complex and diverse smart contracts. To tackle these challenges, we observe that deposit transactions exhibit two distinct features: one is the *functional* features of transaction input data, and the other is the *structural* features of the token-aware call graph. Those two features will be fed to train a classifier to identify deposit transactions.

5.1.1 Transaction Input Data Mining

The most straightforward way to understand a user's deposit transactions is to start by analyzing the interaction transactions between the user and the bridge contract. We first investigate the deposit functions and parameters of different bridges with three bridges as an example, as shown in Fig. 4. Different deposit functions for different bridges essentially inform the bridge contract through a transaction to transfer to other blockchains by a certain amount and asset type. To address this problem, we intend to apply natural language processing (NLP) to characterize the functional characteristics of bridge transactions. Using NLP tools, the semantic similarities of function definition of bridge contracts can be extracted as a distinguishable feature. Specifically, CONNECTOR extracts functional features by decoding transaction data and embedding the normalized text sequence.

Input data decoding. The transaction input data are encoded in a hexadecimal string with semantic gaps for human reading and NLP tools. The input data specifies the function to be called (i.e., the function selector) in the first four bytes, followed by a list of encoded parameters (starting with the 5th byte). To decode the input data, CONNECTOR utilizes the open-source web3 package `decode_function_input` method based on the templates provided in the ABI (introduced in §2). The ABI matches the called function and parameters in the transaction data with human-readable interface definitions. Formally, we get $f(< p_i : r_i >)$, where f is the function name, $< \cdot >$ is the list form, p_i is the i-th parameter and r_i is the results of value.

Vocabulary embedding. Next, we convert decoded input data into vectors to text sequences for extracting transaction semantics. We find the parameters in deposit transactions of different bridges have irrelevant variables other than cross-chain metadata, which may affect the effectiveness of semantic extraction and reduce the performance of our deposit identification method. This inspires us to normalize the decoded data of transactions by mapping parameters other than cross-chain metadata to constant variables. Our renaming rules are as follows: 1) Remain the original function name f and ignore all values v_i of the parameters; 2) If the parameter name is similar to the cross-chain metadata element (introduced in §4), rename it as the corresponding element. e.g., rename "fromAsset" as $asset^s$ (See GitHub for more renaming rules). All other extraneous parameter names are normalized to `var_0`, `var_1`, etc., in the order of occurrence.

$$Normalized(f(< p_i : r_i >)) = f, < p_i >$$
$$p_i = \begin{cases} element, & if\ p_i \approx element, element \in \mathbb{M} \\ \texttt{var}, & otherwise \end{cases} \quad (1)$$

Finally, we train a word embedding model word2vec [12] to get functional features. For the normalized transaction set $S = \{s\}, s = f\ or\ p_i$, we get the word embeddings by $W = Word2Vec(S)$. The word2vec in CONNECTOR is trained with dimension of word vector `vector_size=6`. By converting each normalized text to word embedding, CONNECTOR extracts the functional feature v^{func}_{txhash} of each transaction with a fixed length of 48 (features with insufficient length are filled with zeros). To this end, the functional features with semantics are extracted by CONNECTOR for deposit behavior identification.

5.1.2 Token-aware Call Graph Mining

In DeFi bridge apps, different components are deployed on different blockchains with their scope of work and communicate via function calling. The on-chain router smart contracts are responsible for user interactions and various token contracts [24]. In this part, we explore the call relationships between users and contracts, which may give rise to distinctive structural features for deposit behavior identification.

To accomplish users' deposit behavior, the bridge contracts need to trigger several internal function calls to transfer tokens (i.e., assets) between contracts afterwards. Fig. 5 shows the function call relationship of an example deposit transaction *0xa7ef*. in Poly Network bridge. 1) The user first conducts an external transaction to call function `lock()` of contract `PolyWrapper`; 2) Within contract `PolyWrapper`, the token transfer function `safeTransferFrom()` is called internally; 3) After that, another contract `LockProxy` of the

```
//PolyWrapper.sol
function lock(address fromAsset, uint64 toChainId, bytes
    memory toAddress, uint amount, uint fee, uint id)
    external payable nonReentrant whenNotPaused { //[1]
    external transactions
    ...//requirements
    IERC20(fromAsset).safeTransferFrom(msg.sender,
        address(this), amount); //[2] ERC20 token
        transfer
    amount = _checkoutFee(fromAsset, amount, fee);
    address lockProxy = _getSupportLockProxy(fromAsset,
        toChainId);
    ...//approves
    require(lockProxy.lock(fromAsset, toChainId,
        toAddress, amount), "lock erc20 fail"); // Call
        the lock function of contract lockProxy
    ...}
//LockProxy.sol
function lock(address fromAssetHash, uint64 toChainId,
    bytes memory toAddress, uint256 amount) public
    payable returns (bool) {
    ...//requirements
    require(_transferToContract(fromAssetHash, amount), "
        transfer asset from fromAddress to lock_proxy
        contract  failed!"); //[3] ERC20 token transfer
    ...//cross-chain manager called }
```

Figure 5: The simplified implementation of triggered contracts and functions corresponding to the example transaction *0xa7ef*.

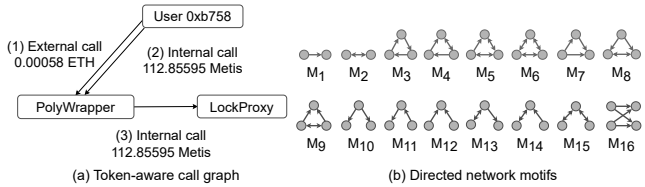


Figure 6: (a) The user *0xb758* transferred 0.00058 ETH to call the contract `PolyWrapper` in the external transaction; Then the token transfer function is called while 112.8556 Metis Token is transferred to contract `PolyWrapper`; Finally, those tokens are further transferred to contract `LockProxy` via its token transfer function. (b) M_1 and M_2 are all connected two-vertice motifs; M_3 – M_{15} are all 13 connected three-vertice motifs; M_{16} is the four-vertice bi-fan motif.

Poly Network bridge is triggered at the entry of the function `lock()`, which subsequently calls token transfer function `_transferToContract()` to further transfer token to the contract `LockProxy`. These call relationships in the bridge transaction can be modeled and mined to find more general features for deposit transaction identification.

Modeling of token-aware call graph. We model the call relationships among each transaction as a call graph. In particular, we focus on token-aware call relationships, since deposit behaviors of cross-chain transactions inevitably involve asset transfers. Token-aware call relationship is denoted as (*caller*, *callee*, *call type*, *tokens*). We take the Ethereum deposit transaction *0xa7ef* in Poly Network as an example, and its token-aware call graph is visualized in Fig. 6(a). The token-aware call graph of this transaction contains three adresses {*0xb757*, *PolyWrapper*, *LockProxy*} and three call relationships {(*0xb757*, *PolyWrapper*, *external call*, 0.00058 *Ether*), (*0xb757*, *PolyWrapper*, *internal call*, 112.86 *Metis*), (*PolyWrapper*, *LockProxy*, *internal call*, 112.86 *Metis*)}.

Motif of token-aware call graph. With a constructed token-aware call graph, we extract the structural features to characterize call relationship patterns for each transaction. Network motifs are high-order network organizations, which is an effective complex network tool that can reflect and extract hidden information of the network's structure [1]. Fig. 6(b) shows 16 network motifs consisting of two and three vertices and a special four-vertice motif. These motif patterns have been extensively explored in previous research and have been demonstrated to unveil the distinctive attributes of diverse networks. Inspired by MoTs [21], a generic transaction semantic representation, we apply the concept of network motifs to characterize the transaction patterns of the token-aware call relationship patterns. For each transaction *txhash*, a 16-dimensional feature vector v^{struc}_{txhash} is extracted. The *i*-th element of v^{struc}_{txhash} means the frequency of the *i*-th network motifs (shown in Fig. 6(b)) in a constructed call graph. We calculate the directed motif M_1-M_{16} by by subgraph matrix computation [1, 21]. To this end, the structural features with call relationships are extracted by CONNECTOR for deposit behavior identification.

5.1.3 Summary for S1

Two distinct features, the functional feature of transaction input data and the structural features of the token-aware call graph, are concatenated to get a more precise and general characterization. Note that input data rather than logs are decoded for deposit identification because the former requires less time and storage. With the extracted features, machine-learning classification tools are employed to identify the deposit behaviors of bridge apps. Thus, the accurate deposit transactions $\mathbb{M}.txhash^s$ of metadata are identified. Also, $\mathbb{M}.sender$, $\mathbb{M}.timestamp^s$, $\mathbb{M}.asset^s$ of metadata can be extracted from the deposit transaction data and trace.

5.2 Withdrawal Transaction Matching

As mentioned in §1, inter-blockchain communication within DeFi bridges is executed through distributed smart contracts rather than centralized institutions. Consequently, semantic gaps exist when association transactions within DeFi bridges. To address this challenge, we analyze the execution logs of bridge contracts to identify withdrawal transactions. Specifically, we employ syntax-semantic parsing on deposit transaction logs to extract cross-chain metadata. Subsequently, we identify the target withdrawal transactions based on the business logic of the bridges.

5.2.1 Syntactic-Semantic Log Parsing

The contract log serves as communication proof between the source chain and the destination chain, encompassing vital information pertaining to cross-chain metadata. In a DeFi bridge, the general flow of the message-passing mechanism is as follows: when the DeFi bridge receives a cross-chain request, the router contract on the source chain calls the token contract to lock the user's tokens. During this call, the token contract triggers a lock action and passes the user's tokens to the router contract. Subsequently, the router contract generates a deposit event E containing detailed information about the locking behavior, including information about the asset type and amount, as *proof* of the locked asset. An example of a deposit event of Poly Network bridge is:

```
event LockEvent(address fromAssetHash, address
    fromAddress, uint64 toChainId, bytes toAssetHash,
    bytes toAddress, uint256 amount);
```

Next, the off-chain relayers will parse the event and confirm the locking operation on the source chain, then authorize the unlocking operation and send the transaction to the router contract on the destination chain. The router contract will then invoke the target token contract and unlock the asset to the user's address on the destination chain.

Syntactic parsing. To bridge the semantic gap, CONNECTOR first conducts log syntax parsing to recover the encoded transaction logs. In particular, CONNECTOR utilizes the `eth_getTransactionReceipt` function of the source chain to retrieve the receipt of a deposit transaction. Then, CONNECTOR analyzes the logs to detect the events emitted during the execution of deposit transactions. The structured log list is parsed by CONNECTOR from transaction receipts based on ABIs, which contains all the event interface definitions (i.e., names and parameters). However, the structured log list itself is undirect to semantic cross-chain metadata information, thus requiring semantic parsing.

Semantic parsing. Semantic parsing aims to map the structured log lists of deposit transactions to cross-chain metadata. According to whether the logs meet the conditions of event name and parameters shown in Table 3, CONNECTOR maps the log lists into the semantic cross-chain transaction metadata. `Decimal()` maps the value to the token amount according to different token decimals. DeFi bridge apps usually support various tokens, thus it is necessary to unify the value of different tokens for further matching. `ID2Chain()` maps the blockchain ID of different bridges to the blockchain names according to their open document. This is typically seen in bridges that support cross-chaining between multiple blockchains. Take the deposit transaction *0xa7ef* in Poly Network as an example. Based on the log list, CONNECTOR locates the event mentioned in the condition and maps the parsed parameters to the metadata. Metadata ($\mathbb{M}$) includes the receiver address on the deposit transaction $\mathbb{M}.receiver$, the sent asset hash $\mathbb{M}.asset^s$, the sent amount $\mathbb{M}.amount^s$, the destination chain $\mathbb{M}.chain^d$, and the timestamp of deposit transaction $\mathbb{M}.timestamp^s$. Combining the metadata given by S1, the cross-chain metadata of deposit transaction *0xa7ef* for further matching is:

$$\mathbb{M} = \begin{cases} sender & = & 0xb758 \\ timestamp^s & = & 1680341603 \\ asset^s & = & 0x9E32(token) \\ amount^s & = & 112.855947137612614726 \\ chain^d & = & Binance\ Smart\ Chain \\ asset^d & = & 0x9E32(token) \\ receiver & = & 0xb758 \end{cases}$$

In this way, CONNECTOR provides cross-chain transaction metadata containing information of the destination chain by syntactic-semantic log parsing.

5.2.2 Business Logic based Target Discovery

In this part, we intend to discover the target withdrawal transaction based on the business logic of bridges. We first implement an efficient search space constructor to obtain candidate transactions, and then design a heuristic matcher among those candidate transactions to discover the withdrawal transaction.

Search space constructor. The search space is constructed based on the assumption that the cross-chain transactions are usually finished soon in a short time interval, which is the business logic of bridges. First, CONNECTOR uses the most recent block number corresponding to $\mathbb{M}.timestamp^s$ and $\mathbb{M}.timestamp^s + \Delta$ (Δ denotes the maximum time interval) as the starting block number and the end block number of the search, respectively. Then, the search space is constructed by CONNECTOR crawling all transactions related to $\mathbb{M}.receiver$ that occur between this interval on $\mathbb{M}.chain^d$. In this process, different types of bridges can choose different Δ, and a detailed discussion of Δ is given in §6.2.

Heuristic matcher. With the obtained search space, CONNECTOR then discovers the target withdrawal transaction within the search space with the following rules:

RULE 1 **(Asset type matching).** Based on $\mathbb{M}.asset^s$and $\mathbb{M}.asset^d$, CONNECTOR filters candidates with matching asset types. That is, if and only if $\mathbb{M}.asset^s$ and $\mathbb{M}.asset^d$ are the same, the transaction type matching is satisfied.

RULE 2 **(Transaction time matching).** Based on the business logic of cross-chain transactions claimed by bridge apps, withdrawal transactions usually finish within 30 minutes (or less) after the deposit transaction is confirmed. Based on $\mathbb{M}.timestamp^s$ and $\mathbb{M}.timestamp^d$, CONNECTOR filters candidates whose time constraint τ is smaller than the threshold. Initially, CONNECTOR takes $\tau = 30$ as the time-matching threshold and adds 10% when there are no candidates. Continuing in this manner, the process will iterate until there are transactions that meet the condition.

RULE 3 **(Transaction amount matching).** Based on the business logic of cross-chain transactions, the cross-chain transaction fee usually accounts for about 0%-3% of the deposit transaction amount.According to $\mathbb{M}.amount^s$ and

Table 3: The mapping of transaction logs and cross-chain transaction metadata in the semantic parsing.

Bridge	*Conditions (event + parameter mapping)*	*Metadata* ($\mathbb{M}$)
Celer cBridge	(Exist `Send` event E and $E.sender = \mathbb{M}.sender$ and $E.receiver = \mathbb{M}.receiver$ and $E.token = \mathbb{M}.asset^s$ and `Decimal`$(E.amount) = \mathbb{M}.amount^s$ and `ID2Chain`$(E.dstChainId) = \mathbb{M}.chain^d$) or (Exist `Deposited` event E and $E.depositor = \mathbb{M}.sender$ and $E.mintAccount = \mathbb{M}.receiver$ and $E.token = \mathbb{M}.asset^s$ and `Decimal`$(E.amount) = \mathbb{M}.amount^s$ and `ID2Chain`$(E.mintChainId) = \mathbb{M}.chain^d$) or (Exist `LogNewTranserOut` event E and $E.sender = \mathbb{M}.sender$ and $E.dstAddress = \mathbb{M}.receiver$ and $E.token = \mathbb{M}.asset^s$ and `Decimal`$(E.amount) = \mathbb{M}.amount^s$ and `ID2Chain`$(E.dstChainId) = \mathbb{M}.chain^d$)	$\mathbb{M}.sender$, $\mathbb{M}.receiver$, $\mathbb{M}.asset^s$, $\mathbb{M}.amount^s$, $\mathbb{M}.chain^d$, $\mathbb{M}.asset^d$ $*$
Multichain	Exist `LogAnySwapOut` event E and $E.from = \mathbb{M}.sender$ and $E.to = \mathbb{M}.receiver$ and $E.token = \mathbb{M}.asset^s$ and `Decimal`$(E.amount) = \mathbb{M}.amount^s$ and `ID2Chain`$(E.toChainID) = \mathbb{M}.chain^d$	
Poly Network	Exist `LockEvent` event E and $E.fromAddress = \mathbb{M}.sender$ and $E.toAddress = \mathbb{M}.receiver$ and $E.fromAssetHash = \mathbb{M}.asset^s$ and $E.toAssetHash = \mathbb{M}.asset^d$ and $E.amount = \mathbb{M}.amount^s$ and `ID2Chain`$(E.toChainId) = \mathbb{M}.chain^d$	

$*$ denotes the metadata element is optional in deposit event logs.

$\mathbb{M}.amount^d$, CONNECTOR calculates the proportion of transaction fee δ through $\delta = \frac{\mathbb{M}.amount^s - \mathbb{M}.amount^d}{\mathbb{M}.amount^s}$, and select candidates whose $\delta \in [0\%, 3\%]$. Similar to RULE 2, due to the special situation where δ may be greater than 3%, CONNECTOR uses 3% as the lower limit of δ and 100% as the upper limit of δ to gradually relax the amount constraint.

In most cases, after executing the above matching rules in sequence, the number of candidate withdrawal transactions obtained is 1. If there is no candidate, the search space can be further expanded by increasing the Δ. If there is more than 1 candidate, the constraint effect can be enhanced by lowering the τ of RULE 2. Finally, CONNECTOR outputs a matching pair of deposit transactions and withdrawal transactions.

5.2.3 Summary for S2

After syntactic-semantic log parsing, CONNECTOR reveals the cross-chain metadata information benefit from the message-passing mechanism of DeFi bridges. Note that logs instead of input data are parsed for withdrawal transaction matching because the former contains richer essential metadata. With the metadata information, the search space of the destination chain can be purposefully constructed and discover the target withdrawal transactions with heuristic rules.

6 Evaluation Results

Existing mainstream mechanisms for cross-chain bridge include: *Hash Time Lock Contract (HTLC)*, *Notary Mechanism*, and *Relay/Relay chain* [14]. For these mechanisms, we select representative bridges based on the following criteria and gather contract and transaction data as our primary sources for experimentation. 1) the top 12 bridges with the highest liquidity in Q1 2023 [10]; 2) available bridge explorer service for constructing ground truth dataset (as investigated in §3); 3) EVM-compatible blockchain support; 4) experienced security incidents. Top 3 bridges that satisfy those conditions: Celer cBridge for HTLC, Multichain for Notary Mechanism, and Poly Network for Relay/Relay chain. Note that CONNECTOR discussed in this study is applicable to other contract-based, non-privacy preserving, EVM compatible DeFi bridges. We then collect contracts and transactions of those bridges.

Table 4: Data (Ethereum) for deposit behavior identification.

Bridges	# Contracts	# Deposit Txs	# Non-deposit Txs	Sum
Celer cBridge	4	7,171	12,823	19,994
Multichain	10	15,483	14,512	29,995
Poly Network	3	19,242	148	19,390

Firstly, our experiments are based on Ethereum as the source chain, since Ethereum is the largest permissionless blockchain platform supporting smart contracts. Ethereum contract addresses are obtained from the bridges' official websites. Then we collect contract transactions from Sept. 2020 to Apr. 2023. Based on collected transactions, we analyze and label the categories, which are divided into deposit and non-deposit transactions. The data statistics are shown in Table 4. Next, we construct a large-scale dataset to obtain the ground truth of cross-chain transaction association. Based on the deposit transactions, we query the bridge's official explorer to obtain the corresponding withdrawal transactions. We conduct a statistical analysis of the blockchains that had the highest transaction volume in common. The results show that the Top 3 blockchains are Ethereum (ETH), Binance Smart Chain (BSC), and Polygon (MATIC). Thus, we will evaluate the cross-chain CONNECTOR from Ethereum to Binance Smart Chain and Polygon.

6.1 Results of Deposit Identification (S1)

In this part, we first evaluate the effectiveness of CONNECTOR in identifying deposit transactions. To explore generalization, we partition the data from the three bridge contracts into training and testing sets. The testing bridge is *not* available during the training process, and the training set *only* contains data from the remaining two bridges. For instance, we train the transaction data of Celer cBridge and Multichain (19,994+29,995=49,989) to test the transaction data of Poly Network (19,390). The proportion of training and testing sets is 72:28. To validate the effectiveness of the extracted features, we employ several classical classifiers in the downstream process and quantify the identification performance using accuracy. These classifiers include Logistic Regression (LR), SVC, Decision Tree (DT), Random Forest (RF), AdaBoost, and Gaussian NB (GN), implemented via scikit-learn [13]. To assess the importance of features, we separately implement: 1) using only functional features; 2) using only

Table 5: Results in deposit transaction identification.

Testing	Features	Evaluating Classifiers (Accuracy %) LR	SVC	RF	AdaBoost	GN	Avg.
Poly Network†	Struc.	72.97	75.16	75.16	75.16	72.97	74.43
	Func.	100	100	50.11	100	48.94	83.18
	Struc. + Func.	100	100	99.98	100	100	99.97
Multichain†	Struc.	72.51	74.72	74.72	74.72	72.51	73.98
	Func.	100	83.18	51.69	99.85	51.68	81.04
	Struc. + Func.	100	100	100	100	100	100
Celer cBridge†	Struc.	72.08	74.74	74.74	74.74	72.08	73.85
	Func.	89.87	89.87	74.51	74.51	84.76	81.34
	Struc. + Func.	100	100	100	100	99.97	99.99

† Among those three main bridges, one serves as the testing while the others as training.

Testing	Features	Evaluating Classifiers (Accuracy %) LR	SVC	RF	AdaBoost	GN	Avg.
Other 10 Bridges‡	Struc.	75.00	79.00	79.00	79.00	75.00	77.67
	Func.	42.50	47.00	51.00	48.00	39.50	43.75
	Struc. + Func.	99.00	99.00	100	100	87.50	97.58

‡ Randomly select ten bridges as testing while the main three bridges serve as training.

structural features; and 3) using both structural features and functional features (i.e. default CONNECTOR).

Table 5 demonstrates the experimental results. CONNECTOR successfully identifies almost 100% deposit behavior on average using both structural and functional features. With both features, the accuracy variance of our approach among different classifiers is close to 0. CONNECTOR ultimately adopts the AdaBoost classifier [6], which performs best on both structural and functional features. Using only structural features, it can only identify 74.43% deposit behavior on average, with stable performance. Using only functional features, it can identify deposit behavior more accurately but is unstable with higher variance. We will explain the possible reasons below. Furthermore, to demonstrate the versatility of CONNECTOR across a wide range of DeFi bridges, we conduct an additional experiment by collecting 10 additional contracts and transactions for different Bridges. We randomly collect 10 deposit and 10 non-deposit transactions for each additional bridge. We use the original data from the three bridges as the training set and Table 5 demonstrates the results. Based on the structural features alone, it is stable and can still identify 77% of deposit transactions. The functional-based method can only identify 44% of the deposit behavior. Our CONNECTOR combining both features can still maintain the accuracy rate of 87.5% to 100% in a wider range of bridges. The results indicate that CONNECTOR can accurately and stably identify deposit behavior. CONNECTOR combining both features excels by combining explicit semantic features – functional features from input data (ensuring accuracy in similar scenarios), with implicit semantic features – structural features from call relationships (ensuring generality in dissimilar cases), which enhances overall identification performance.

6.2 Results of Withdrawal Matching (S2)

We evaluate the effectiveness in matching withdrawal transactions with the following metrics: 1) Matching Rate (MR). The percentage of associated results that match ground truth labels. The higher rate means a better traceability of bridge apps. 2) Zero Hit Rate (ZHR). The percentage of results that no matching candidate transactions are found. 3) Wrong Hit Rate (WHR). The percentage of results that fail to locate the correct one among candidate transactions.

Table 6: Results in withdrawal transaction matching.

Bridges	Ethereum ⇒ BSC MR	ZHR	WHR	# All	Ethereum ⇒ Polygon MR	ZHR	WHR	# All
Celer cBridge	95.35%	4.06%	0.59%	7,296	95.65%	2.68%	1.67%	598
Multichain	98.99%	0.71%	0.30%	8,349	94.08%	4.24%	1.68%	2,144
Poly Network	92.55%	6.70%	0.75%	5,460	92.66%	6.79%	0.55%	545
Other 10 Bridges	96.72%	1.24%	2.04%	3,965	96.24%	1.75%	2.01%	1,144

Table 6 demonstrates the experimental results of CONNECTOR in the withdrawal transaction matching. CONNECTOR successfully associates most of the transactions (more than 92%) of different bridges among 21,105 transactions to Binance Smart Chain and 3,287 transactions to Polygon. In particular, nearly 98.99% of the Multichain withdrawal transactions could be tracked. As described in §3, there are 13 bridges that provide cross-chain transaction tracking services. To illustrate CONNECTOR's effectiveness outside these three bridges, we also collect cross-chain transaction pairs for the remaining 10 bridges and find that 96% of the transactions are successfully matched.

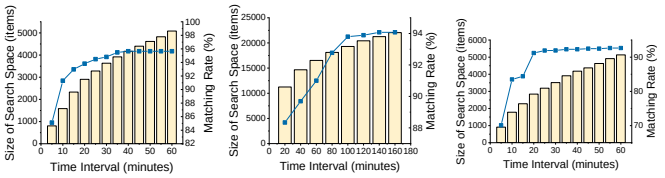
Also, we construct the search space at several different time intervals and record the corresponding search space size and matching rate. The results are shown in Fig. 7. The increasing time interval led to increasing search space and CONNECTOR's matching rate (reducing the zero hits mentioned in the previous paragraph). The matching rate increases faster when the time interval is small, but the matching rate stabilizes later. If the increase in time interval does not increase the matching rate, the inflection point is used to set the time interval. Note that the experimental results of CONNECTOR report the optimal time interval Δ in the search space constructor.

6.2.1 Compared with Methods for CeFi Bridges

Note that the existing methods are not supported for direct use on DeFi bridges. **YKM Heuristic [23, 25]** is designed for CeFi bridges to directly obtain the cross-chain deposit transactions and withdrawal transactions via off-chain centralized information. YKM Heuristic syncs all on-chain data and uses a time constraint to acquire all transactions as the deposit transaction search space, relying on centralized APIs to directly return the matching withdrawal transaction hashes (as described in Table 2 and motivation in §3). However, the centralized API of the bridge is against the decentralized nature of the DeFi bridge, thus the assumption in our problem is not API dependent (See §4). To meet the scenario of DeFi bridges, some adaptive changes are made for experiments: 1) filter the transactions of bridge contracts instead of syncing all data to speed up the experiments; 2) no internal APIs are provided in DeFi bridges, only heuristic rules. These changes have also

Table 7: MR results of withdrawal transaction matching.

Methods	ETH ⇒ BNB			ETH ⇒ Polygon		
	Celer cBridge	Multichain	Poly Network	Celer cBridge	Multichain	Poly Network
YKM [23]	32.50%	0.70%	2.30%	61.04%	7.56%	1.28%
S1 × YKM [23]	36.39%	4.54%	1.25%	60.87%	21.04%	1.28%
CONNECTOR	95.35%	98.99%	92.55%	95.65%	94.08%	92.66%
Time Interval (Δ)	90	120	80	40	140	55



(a) Celer cBridge (b) Multichain (c) Poly Network

Figure 7: The effect of time interval Δ on matching rate and search space size (Ethereum ⇒ Polygon). Inflection points of Δ in Celer cBridge, Poly Network, and Multichain are 40, 140, and 55, respectively.

been applied by the authors [23]. **S1 × YKM Heuristic** is a variant of CONNECTOR. Among transactions of bridge apps, the deposit and withdrawal transactions of bridge contracts are identified by our identification step. With the identified transactions, the YKM Heuristic is used instead of our matching step. The comparative results are shown in Table 7. Different methods are set with the same Δ parameter under the same bridge to ensure fairness. The YKM Heuristic associates a minority of the cross-chain transaction pairs because it acquires a larger set of candidate transactions with similar time and amount but can not handle them. This result is consistent with the authors' findings [23] that the matching rates ***"decreased significantly"*** without the usage of APIs. The YKM Heuristic adding identification method also lacks the ability to track cross-chain transactions in DeFi bridges, because the message between bridge contracts remains unknown without log mining. Under a similar time constraint, we add receiving address constraint and token amount to narrow the search space, supporting ERC20 tokens and eliminating reliance on CeFi bridge APIs. The results indicate the effectiveness and necessity of our log-based matching method in CONNECTOR.

6.2.2 Compared with Explorers of DeFi Bridges

In our practical experiments, we observe the superiority of our CONNECTOR over bridge explorers. We conduct large-scale experiments on three bridges that offer cross-chain transaction tracking services to obtain the ground truth. Particularly, we input the deposit transactions into the bridge browser and crawl the destination chain transactions returned. However, we find that some deposit transactions yield results in our CONNECTOR, while the bridge tracking service fails to return any results. For instance, on Jun-28-2021 11:37:11, a

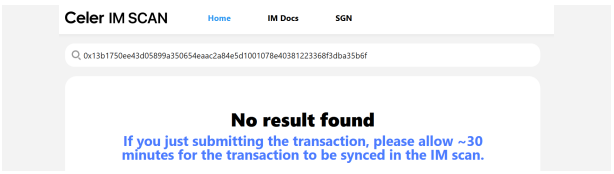


Figure 8: Fail to track and return "No result found".

user on Ethereum initiated a cross-chain transaction to Celer cBridge, with the deposit transaction *0x13b1*. When we query this transaction hash in the bridge explorer, the result returned is "No result found", as shown in Fig. 8. By using CONNECTOR, we find that this deposit transaction involves transferring 15 USDT tokens to Binance Smart Chain, and discover a possible target withdrawal transaction *0x2fda* in which the user received 14.64255 BSC-USD on Jun-28-2021 11:40:42. Upon manually inspecting the open-source documentation and implementation of Celer cBridge, we confirm that transaction *0x2fda* is indeed a withdrawal transaction. This case demonstrates that our CONNECTOR can achieve cross-chain transaction association, even in cases where bridge explorers do not provide support, further highlighting the utility of our CONNECTOR. Furthermore, we observe that this deposit transaction *0x13b1* is executed by one of Celer cBridge's Ethereum contracts *0x841c*. We also randomly select several transactions associated with this contract and none of them obtain the transaction association results from cBridge explorer. The reason may be due to an update of Celer cBridge to Version 2.0[6], rendering the old contract incompatible with the transaction association service.

To compare the efficiency of manual transaction association and CONNECTOR, we track the earliest 50 transactions on Celer cBridge using both methods, with our CONNECTOR and the manual approach conducted by two experienced Ph.D. students familiar with cross-chain bridges. Statistically, each transaction takes approximately 5-10 minutes of manual effort (using blockchain explorers), while our CONNECTOR (in deducting transaction fetching) takes just 0.84 seconds per transaction. This result not only shows that CONNECTOR can enhance cross-chain transaction association results (available in the open repository), but also improves the efficiency of cross-chain transaction association.

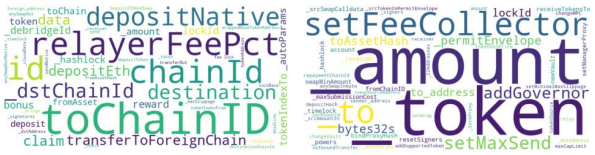
7 Empirical Analysis

In this section, we conduct an empirical analysis of cross-chain transaction pairs based on CONNECTOR's output.

7.1 Analysis of Identified Deposit Transactions

Functional Features. We calculate the frequency of each element defined in §5.1.1 (function name or parameter name),

[6] https://blog.celer.network/2021/11/29/cbridge-2-0-community-driven-upgrade-incoming/



(a) Deposit transactions (b) Non-deposit transactions

Figure 9: Parameter cloud of transactions' input data.

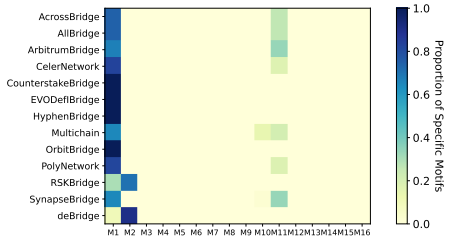


Figure 10: Distribution of the various motifs in deposit transactions of 13 DeFi bridges.

and generate a word cloud map as shown in Fig. 9. It is easy to see that the deposit transactions and non-deposit transactions are obviously different. The two most frequent elements of non-deposit transactions are related to the semantics of "amount" and "token", which frequently appear in token transfer transactions in normal DApps. Clearly, the most frequent elements of non-deposit transactions are related to the semantics of "chain" and "relayer", which are consistent with our pre-defined cross-chain metadata in §4. However, extracting semantics directly from transaction function names and parameters provides strong yet limited information. Thus, CONNECTOR works well when deposit functions share similar names and align with metadata.

> **Finding 1.** The functional features of deposit transactions exhibit distinct characteristics, particularly in bridges that employ similar implementation, which enhances the accuracy of identification.

Structural Features. We delve into the structural characteristics of deposit transactions of token-aware call graphs, by counting the numbers of transaction motifs (the motif types are shown in Fig. 6(b). As illustrated in Fig. 10, the subgraph structure of cross-chain transactions predominantly manifests as M_1, M_2, M_{10}, and M_{11}. M_1 represents the transaction pattern where accounts directly send native tokens to the bridge contract. M_2 reveals the process where accounts initiate a cross-chain request, after which the bridge directly transfers tokens to the recipient address. M_{10} describes a more complex procedure where accounts use the bridge contract to convert native tokens into other tokens and complete the cross-chain deposit. In this process, the cross-chain routing contract exchanges specified tokens with the token contract and deposits the tokens into the cross-chain bridge token contract, thereby finalizing the deposit. However, call relationships may also be present in non-deposit transactions, reducing their significance. Nevertheless, its implicit nature makes it less susceptible to variations in function implementations.

Table 8: The number of matched transaction pairs that use the same/different account for cross-chain.

Bridge	# $Pair_{same}$	# $Pair_{diff}$	# Unique	# Cluster
Allbridge Core	501 (95.79%)	22 (4.21%)	20 (90.91%)	20 (90.91%)
Celer cBridge	7,884 (99.87%)	10 (0.13%)	2 (2%)	2 (2%)
Connext Bridge	535 (96.40%)	20 (3.60%)	7 (35%)	7 (35%)
Debridge	44 (91.67%)	4 (8.33%)	4 (100%)	3 (75%)
Multichain	10,253 (97.71%)	240 (2.29%)	177 (73.75%)	150 (62.5%)
Poly Network	5,995 (99.83%)	10 (0.17%)	6 (60%)	6 (60%)
Router Protocol	17 (100.00%)	0 (0.00%)	-	-
Stargate	1,210 (99.84%)	2 (0.16%)	2 (100%)	2 (100%)
Symbiosis	148 (98.01%)	3 (1.99%)	2 (66.67%)	2 (66.67%)
Synapse Protocol	677 (98.26%)	12 (1.74%)	12 (100%)	12 (100%)
Transit Swap	471 (33.22%)	947 (66.78%)	688 (72.65%)	630 (66.53%)
Wormhole	476 (98.14%)	9 (1.86%)	5 (55.56%)	5 (55.56%)

> **Finding 2.** Structural features of token-aware call graphs in deposit transactions contain the main procedure of DeFi bridges in general, but not significantly enough.

7.2 Analysis of Matched Transaction Pairs

We explore the cross-chain transaction pairs output by CONNECTOR, including issues about user anonymity protection, time efficiency, and economic benefits.

User Anonymity Protection. We analyze the deposit account ($\mathbb{M}.sender$) and the withdrawal account ($\mathbb{M}.reciever$) of matched cross-chain transaction pairs, as shown in Table 8. Except for the Poly Network bridge, in the vast majority of cross-chain transactions, the sender of the deposit transaction and the receiver of the withdrawal transaction are both at the same address, and these transactions have unique addresses ('# Unique' column). Especially for the Router Protocol bridge, the ratio of transactions that use the same address is 100% (Assume that one entity uses only one account [23].) This suggests that most cross-chain bridge users are not aware of privacy protection. Very few users use the new address as the recipient, and a deliberate effort to enhance anonymity could be involved in illegal financial activities such as money laundering (discussed in §5). Further, ours performs further clustering analysis on cross-chain transaction pairs that employ different addresses ('# Cluster' column). The clustering strategy groups accounts within the minimum connectivity graph into one class of accounts. We find that this portion of privacy-enhancing cross-chain transactions mainly comes from different users.

> **Finding 3.** Most cross-chain bridge users lack the awareness of anonymity privacy protection, using the withdrawal address of destination chains as same as the deposit account of source chains.

Time Efficiency. To explore the time efficiency of bridges across chains, we statistically analyze the deposit and withdrawal time intervals for each cross-chain transaction pair. According to Fig. 11(a), we find that the vast majority of the destination chain's withdrawal transactions are able to be completed within 30 minutes, which validates the rationale for initially setting the business rule's time constraint τ to less than 30 minutes at our methodology level (Review details in RULE 2 of §5.2.2). Further observation reveals that specific cross-chain bridges, such as DeBridge and Allbridge, perform better in terms of efficiency, with most of the transactions they support being completed quickly within 5 minutes. In contrast, other cross-chain bridges, such as Wormhole and Symbiosis, are mostly spread out in the 10 to 30-minute range. These reveal that there are some differences in the performance of different cross-chain bridges. The bridges encounter network congestion on the blockchain or additional validation time required when executing cross-chain transactions. In other cases, the user initiates more than one transaction of similar amounts simultaneously, which may result in wrong hits. Fortunately, these cases are unusual since most users make cross-chain transactions one at a time and on demand.

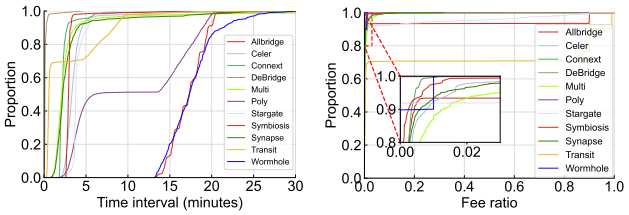
> **Finding 4.** The vast majority of withdrawal transactions can be completed in less than 30 minutes on destination chains, but there may be instances where extra time intervals are required.

Economic Benefits. To explore the economic benefits of cross-chain bridges, we statistically analyze the handling fee ratio for each cross-chain transaction pair, and obtain the results shown in Fig. 11(b). The statistics show that 90% cross-chain transaction pairs have a fee ratio of less than 1%, while our amount constraint δ is set to 3% (Review details in RULE 3 of §5.2.2). However, we also observe that some of the transactions have fees that exceed the constraints, and speculate that these accounts may be willing to pay higher fees to speed up the process due to the urgent need for transaction speed. Note that we dynamically adjusted the detection range of the amount constraint δ to address the possible high fee share cases and improve the generality.

> **Finding 5.** Most cross-chain transaction pairs earn a fee ratio of less than 1%. Users sometimes speed up cross-chain transactions by increasing transaction fees.

7.3 Potential Use in Anti-money Laundering

To verify the practical value of CONNECTOR in maintaining DeFi financial security, we use CONNECTOR to supplement cross-chain money laundering (ML) transactions for the security of decentralized finance. Existing ML datasets, such as Elliptic (Bitcoin ML dataset) [18] and *EthereumHeist* (Ethereum ML dataset) [19], only identify ML transactions on single blockchain and do not include cross-chain transactions. First, we filter out bridge service contracts in the *EthereumHeist* dataset [19] and find four security incidents, including: *Akropolis Hack*, *Coinrail Hack*, *Fake_Phishing5041*, and *Kucoin Hacker*Then, for each security incident, we identify the cross-chain deposit ML transactions and obtain 5 deposit ML transactions for *Akropolis Hack*, 5 for *Coinrail Hack*, 1 for *Fake_Phishing5041*, and 15 for *Kucoin Hacker*. We further utilize the matching module of CONNECTOR to discover related withdrawal transactions on the destination chain. Thus, with the use of our tool, security companies and regulatory authorities can supplement the cross-chain transfer paths of funds involved in these cases.



(a) Time interval of transaction pairs (b) Fee ratio of transaction pairs

Figure 11: Cumulative distribution of cross-chain transaction pairs among 13 DeFi bridges.

8 Threats to Validity

Internal validity. The bridge smart contract and deposit transaction labels for cross-chain transaction association may not be fully completed and reliable. To mitigate the issue, two of the authors independently collected contracts and labeled the deposit transactions with the help of the third author to resolve a possible disagreement. Also, bridges with totally disruptive function designs may be limited in the accuracy of learning-based identification. To mitigate this issue, structural features extracted from the token-aware call graph can make a complement, with accuracy stabilized at about 75%. The EOA-based CeFi bridge may exhibit unpredictable results without the bridge's APIs. The impact is limited because the percentage of CeFi bridges is quite small (as described in §3). The transaction tracking for inconsistent deposit-withdrawal transactions [24] and many-to-many cross-chain relationships are not in the CONNECTOR's scope. Those two issues belong to the research of attack detection [27] and coin mixing tracking [20], respectively. Furthermore, our research does not encompass non-fungible token (NFT) bridges, privacy-enhancing bridges, or non-EVM-compatible bridges, which warrant investigation in future studies.

External validity. CONNECTOR's effectiveness largely depends on the availability of open-source contract codes. Bridges lacking these codes may pose challenges for our method. Among the contract-based DeFi bridges, 79.35% of bridges disclose their contract addresses or codes (as investigated in §3). Meanwhile, developers are usually willing to

open their code so that whitehats can look for vulnerabilities.

9 Related work

Tracing cross-chain transactions. Bridging apps enable transfers of crypto assets across chains, making the routes of funds more difficult to trace. Tracing cross-chain transactions therefore becomes important for needs such as anti-money laundering and increased transaction transparency. Yousaf *et al.* [23] is the first to utilize the internal API and heuristic matching rules to achieve bridge traceability. Zhang *et al.* [25] devised CLTracer, a framework grounded in address relationships for cross-chain transaction clustering. Existing methods cannot be directly applied to DeFi bridge traceability without centralized ledgers or APIs.

Blockchain transaction analysis. Smart contracts execute their functions by receiving transactions that include different actions and semantics. Currently, there is a lot of research focused on the transactions and behaviors of smart contracts [2, 16]. For example, Wang *et al.* [17] used a sequence-based method to detect attack transactions in DeFi. Wu *et al.* [21] proposed a transaction semantic graph for transaction identification. Zhou *et al.* [26] identified inconsistent behaviors of DApps by contrasting the behaviors from front-end, wallet, and smart contract.

10 Conclusion

In this paper, we propose CONNECTOR, the first automatic transaction association tool designed for contract-based DeFi cross-chain bridges. CONNECTOR extracts features from transaction data and traces to accurately identify the deposit transactions and mine the executed logs of bridge contracts to match the correct withdrawal transactions. We build the first dataset with 24,392 cross-chain transaction pairs from bridge apps and conduct experiments. CONNECTOR successfully identifies 100% deposit transactions, associates 95.81% withdrawal transactions, and is superior to the bridge official website in practical use. Furthermore, CONNECTOR is able to associate additional cross-chain transactions than bridge official explorers, which demonstrates the superiority of CONNECTOR. Based on the association results, we explore the issues about user privacy protection, time efficiency, and economic benefits of the cross-chain bridges, and we find that DeFi bridges are abused in crypto money laundering. In summary, CONNECTOR enables bridge developers, regulators, and users to enhance bridge traceability in the multi-chain DeFi ecosystem.

References

[1] Austin R. Benson, David F. Gleich, and Jure Leskovec. Higher-order organization of complex networks. *Science*, 353(6295):163–166, 2016.

[2] Ting Chen, Zihao Li, Xiapu Luo, Xiaofeng Wang, Ting Wang, Zheyuan He, Kezhao Fang, Yufei Zhang, Hang Zhu, Hongwei Li, Yan Cheng, and Xiaosong Zhang. Sigrec: Automatic recovery of function signatures in smart contracts. *IEEE Transactions on Software Engineering*, 48(8):3066–3086, 2022.

[3] Jane Cleland-Huang, Orlena C. Z. Gotel, Jane Huffman Hayes, Patrick Mäder, and Andrea Zisman. Software traceability: Trends and future directions. In *Future of Software Engineering*, page 55–69, 2014.

[4] Elliptic Team. The state of cross-chain crime report. www.elliptic.co/resources/state-of-cross-chain-crime-2023, 2023.

[5] Yuzhou Fang, Daoyuan Wu, Xiao Yi, Shuai Wang, Yufan Chen, Mengjie Chen, Yang Liu, and Lingxiao Jiang. Beyond "protected" and "private": An empirical security analysis of custom function modifiers in smart contracts. In *International Symposium on Software Testing and Analysis*, page 1157–1168, 2023.

[6] Yoav Freund and Robert E Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[7] Jianjun Huang, Songming Han, Wei You, Wenchang Shi, Bin Liang, Jingzheng Wu, and Yanjun Wu. Hunting vulnerable smart contracts via graph embedding based bytecode matching. *IEEE Transactions on Information Forensics and Security*, 16:2144–2156, 2021.

[8] William E Bodell III, Sajad Meisami, and Yue Duan. Proxy hunting: Understanding and characterizing proxy-based upgradeable smart contracts in blockchains. In *USENIX Security Symposium*, pages 1829–1846, 2023.

[9] Jiao Jiao, Shuanglong Kan, Shang-Wei Lin, David Sanan, Yang Liu, and Jun Sun. Semantic understanding of smart contracts: Executable operational semantics of solidity. In *IEEE Symposium on Security and Privacy*, pages 1695–1712, 2020.

[10] Chuks Nnabuenyi Jr. Top 12 bridges with highest liquidity in Q1 2023. https://cryptotvplus.com/2023/05/top-12-bridges-with-highest-liquidity-in-q1-2023/, May 2023.

[11] Sung-Shine Lee, Alexandr Murashkin, Martin Derka, and Jan Gorzny. Sok: Not quite water under the bridge: Review of cross-chain bridge hacks. In *IEEE International Conference on Blockchain and Cryptocurrency*, May 2023.

[12] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *ArXiv Preprint ArXiv:1301.3781*, 2013.

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[14] Kunpeng Ren, Nhut-Minh Ho, Dumitrel Loghin, Thanh-Toan Nguyen, Beng Chin Ooi, Quang-Trung Ta, and Feida Zhu. Interoperability in blockchain: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 2023.

[15] Jianzhong Su, Xingwei Lin, Zhiyuan Fang, Zhirong Zhu, Jiachi Chen, Zibin Zheng, Wei Lv, and Jiashui Wang. Defiwarder: Protecting defi apps from token leaking vulnerabilities. In *International Conference on Automated Software Engineering*, 2023.

[16] Tianle Sun, Ningyu He, Jiang Xiao, Yinliang Yue, Xiapu Luo, and Haoyu Wang. All your tokens are belong to us: Demystifying address verification vulnerabilities in solidity smart contracts. In *USENIX Security Symposium*, pages 3567–3584, 2024.

[17] Bin Wang, Xiaohan Yuan, Li Duan, Hongliang Ma, Chunhua Su, and Wei Wang. Defiscanner: Spotting defi attacks exploiting logic vulnerabilities on blockchain. *IEEE Transactions on Computational Social Systems*, pages 1–12, 2022.

[18] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*, 2019.

[19] Jiajing Wu, Dan Lin, Qishuang Fu, Shuo Yang, Ting Chen, Zibin Zheng, and Bowen Song. Toward understanding asset flows in crypto money laundering through the lenses of Ethereum heists. *IEEE Transactions on Information Forensics and Security*, 19:1994–2009, 2024.

[20] Lei Wu, Yufeng Hu, Yajin Zhou, Haoyu Wang, Xiapu Luo, Zhi Wang, Fan Zhang, and Kui Ren. Towards understanding and demystifying bitcoin mixing services. In *Proceedings of the Web Conference*, pages 33–44, 2021.

[21] Zhiying Wu, Jieli Liu, Jiajing Wu, Zibin Zheng, Xiapu Luo, and Ting Chen. Know your transactions: Real-time and generic transaction semantic representation on blockchain & web3 ecosystem. In *Proceedings of the ACM Web Conference*, page 1918–1927, 2023.

[22] Xiufeng Xu. Towards automated migration for blockchain-based decentralized application. In *International Conference on Software Engineering: Companion Proceedings*, page 155–157, 2020.

[23] Haaroon Yousaf, George Kappos, and Sarah Meiklejohn. Tracing transactions across cryptocurrency ledgers. In *USENIX Security Symposium*, pages 837–850, Aug. 2019.

[24] Jiashuo Zhang, Jianbo Gao, Yue Li, Ziming Chen, Zhi Guan, and Zhong Chen. Xscope: Hunting for cross-chain bridge attacks. In *International Conference on Automated Software Engineering*, oct 2022.

[25] Zongyang Zhang, Jiayuan Yin, Bin Hu, Ting Gao, Weihan Li, Qianhong Wu, and Jianwei Liu. CLTracer: A cross-Ledger tracing framework based on address relationships. *Computers & Security*, 113:102558, 2022.

[26] Jianfei Zhou, Tianxing Jiang, Haijun Wang, Meng Wu, and Ting Chen. Dapphunter: Identifying inconsistent behaviors of blockchain-based decentralized applications. In *International Conference on Software Engineering: Software Engineering in Practice*, pages 24–35, 2023.

[27] Liyi Zhou, Xihan Xiong, Jens Ernstberger, Stefanos Chaliasos, Zhipeng Wang, Ye Wang, Kaihua Qin, Roger Wattenhofer, Dawn Song, and Arthur Gervais. Sok: Decentralized finance (DeFi) attacks. In *IEEE Symposium on Security and Privacy*, pages 2444–2461, 2023.